Learning Day 13

Reinforcement Learning Fundamentals

Notebook Step-by-Step Summary

Ruturaj Mokashi

Purpose	Explain every notebook cell from start to finish: theory, concept, what the code does, what the output means, and how the step supports the Day 13 MVD assignment.
Notebook scope	CartPole random and heuristic policies, FrozenLake tabular Q-Learning, alpha/gamma comparison, slippery FrozenLake, DQN, Double DQN and final MVD coverage.
How to use	Read this together with the notebook. The document follows the notebook from cell 0 to the final cell so you can understand the flow without guessing why a cell exists.

1. Executive overview

This summary document explains the final Day 13 Reinforcement Learning notebook cell by cell. It is written for later revision: each step tells what concept is being used, what the code is doing, and how to interpret the result. The notebook is different from normal supervised learning notebooks because there is no CSV target column. The agent creates training data by interacting with Gymnasium environments.

The notebook has three main practical parts. First, CartPole is used to understand state, action, reward and episode termination. Second, FrozenLake is used to implement tabular Q-Learning with epsilon-greedy exploration. Third, CartPole is used again for Deep Q-Learning, where a neural network replaces the Q-table.

Section	Cells	Main idea
Setup	0-6	Notebook style helpers, dependencies, seeds and reproducibility.
Beginner	7-23	CartPole random policy, heuristic policy, reward comparison and done=True.
Intermediate	24-50	FrozenLake Q-table, epsilon-greedy Q-Learning, alpha/gamma tuning and slippery transitions.
Expert	51-77	DQN, replay buffer, target network, Double DQN and learning-curve comparison.
Final	78-81	MVD checklist and final conclusion.

2. Results snapshot from the executed notebook

These are the main outputs that explain what happened during execution. Because RL uses randomness, exact values can change slightly across runs, but the interpretation remains the same.

Area	Result	Meaning
CartPole random policy	Rewards: 30, 46, 30, 24, 28	Random actions produce unstable survival time.
CartPole heuristic	Mean reward about 43.53 vs random mean about 29.33	The pole-angle rule improves the baseline but is still simple.
done=True example	50 steps, total reward 50, terminated=True	The episode stopped because the environment ending condition occurred.
FrozenLake deterministic	Last-100 mean reward about 0.94, successes 7,655	Q-Learning learned a strong policy when movement is predictable.
Alpha/gamma tuning	Fastest: alpha=0.1, gamma=0.90 at about episode 2,408	Parameter choice affects convergence speed.
FrozenLake slippery	Last-100 mean reward about 0.20, successes 1,704	Stochastic transitions make learning much harder.
Standard DQN	Best reward 169, final 50-episode mean 17.55, >195 not reached	The controlled short run did not solve CartPole.
Double DQN	Best reward 336, final 50-episode mean 46.23, >195 not reached	DDQN performed better in this run, but also did not reach the stable threshold.

3. Cell-by-cell notebook summary

This section follows the notebook from cell 0 to the end. Markdown cells explain the idea and prepare the reader. Code cells create variables, run environments, train agents, print results or produce charts.

Setup and notebook structure

Notebook Cell 0 (Markdown)	**Learning Day 13 — Reinforcement Learning Fundamentals**
Theory / concept	MVD Assignment Notebook: CartPole, FrozenLake, Q-Learning, DQN and Double DQN Ruturaj Mokashi
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 1 (Markdown)	**Notebook purpose**
Theory / concept	This notebook completes the Day 13 MVD assignment. Reinforcement Learning does not start from a labelled CSV file. Instead, an agent interacts with an environment, observes the state, chooses an action, receives a reward, and learns from the result. The notebook is structured as a clean assignment document: first CartPole environment exploration, then tabular Q-Learning on FrozenLake, and finally DQN / Double DQN with PyTorch. Each step includes a simple explanation, small commented code, a chart or table where useful, and an output-specific conclusion.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 2 (Markdown)	**Environment requirement**
Theory / concept	Day 13 uses Gymnasium environments instead of CSV files. The local Python environment must have gymnasium and torch installed. This notebook does not install packages inside the notebook to avoid breaking the kernel. If needed, install once outside the notebook: python -m pip install "gymnasium[classic-control,toy-text]" torch
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 3 (Markdown)	Cell 1 — Create display, table, conclusion and chart helpers
Theory / concept	This cell creates a small presentation layer for the notebook. The most important fix compared with the earlier version is the chart helper: every chart is saved as a PNG and then displayed from the saved image. This makes charts visible after reopening the notebook and avoids broken chart outputs. Tables are displayed in a consistent format. Green conclusion boxes explain the actual output. Charts are saved into a relative folder: outputs/day13_charts .
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 4 (Code)	Cell 1 - DISPLAY HELPERS AND CHART SETTINGS
Theory / concept	This step prepares notebook hygiene. Instead of repeating styling code in every cell, we create reusable display helpers for blue explanation boxes, green conclusion boxes, clean tables and saved PNG charts.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	Notebook display helpers are prepared. Tables, explanation boxes, conclusion boxes and saved PNG chart outputs are available for the rest of the notebook.

Notebook Cell 5 (Markdown)	Cell 2 — Import RL libraries and fix seeds
Theory / concept	This cell imports Gymnasium and PyTorch. Gymnasium provides the CartPole and FrozenLake environments. PyTorch is used later to create the DQN and Double DQN neural networks. We also fix seeds so the results are easier to reproduce. Gymnasium creates the environments. PyTorch trains neural Q-networks. The seed makes random behavior more stable between runs.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 6 (Code)	Cell 2 - IMPORT GYMNASIUM, PYTORCH AND FIX SEEDS
Theory / concept	This step prepares the RL toolkit. Gymnasium supplies environments; PyTorch builds and trains neural networks; fixed seeds make random actions and neural training more reproducible.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The executed environment used Gymnasium 1.3.0, PyTorch 2.2.2, CPU training device and project seed 42. This confirms that the RL environments and PyTorch models can be created reproducibly.

Beginner: CartPole environment exploration

Notebook Cell 7 (Markdown)	**Section 1 — Beginner MVD: Exploring CartPole-v1**
Theory / concept	CartPole is a control environment. The agent sees four observation values and chooses between two actions: push left or push right. The goal is to keep the pole balanced for as many steps as possible. We first test a random policy, then a simple rule-based heuristic.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 8 (Markdown)	**Cell 3 — Create CartPole and explain the observation variables**
Theory / concept	CartPole gives the agent four numbers at every step. These four values describe the physical state of the system. Understanding them is important because the simple heuristic uses the pole angle, and the DQN later uses all four values as neural-network inputs. Cart position: where the cart is. Cart velocity: how fast it moves. Pole angle: how much the pole leans. Pole angular velocity: how fast the pole is falling or recovering.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 9 (Code)	**Cell 3 - CREATE CARTPOLE AND INSPECT OBSERVATIONS**
Theory / concept	This step inspects the first environment. In RL, the observation is the information the agent sees before it acts. CartPole has continuous observations, not table-like rows.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	CartPole returns four observation values: cart position, cart velocity, pole angle and pole angular velocity. In the executed run, the initial values were small, meaning the cart-pole system started close to balanced.

Notebook Cell 10 (Markdown)	**Cell 4 — Run 5 random CartPole episodes**
Theory / concept	The MVD asks for five episodes with random actions. A random policy does not use the state. It simply chooses action 0 or action 1 by chance. This gives us a baseline before using a rule-based or learning-based policy. Reward in CartPole is usually 1 per time step survived. Total reward equals how long the pole stayed balanced. Random policy is expected to be unstable.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 11 (Code)	**Cell 4 - RUN 5 RANDOM CARTPOLE EPISODES**
Theory / concept	This step creates a baseline. A random policy chooses actions without using the observation. Baselines help us judge whether later rules or learning actually improve performance.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The five random-policy rewards were 30, 46, 30, 24 and 28. Because the agent acts randomly, the pole sometimes survives longer and sometimes falls quickly.

Notebook Cell 12 (Markdown)	**Cell 5 — Visualise random-policy rewards**
Theory / concept	A chart makes the random baseline easier to understand. If the bars vary a lot, it means the random policy is unstable. This is expected because the agent is not using the observation variables. Each bar is one random episode. The dashed line is the mean reward. Higher reward means the pole survived longer.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 13 (Code)	**Cell 5 - VISUALISE RANDOM-POLICY REWARDS**
Theory / concept	This step turns the random rewards into a chart so we can see instability. In RL, one number is often misleading because episodes can vary a lot.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The chart shows unstable episode rewards for the random policy. This is expected because no learning or rule is used.

Notebook Cell 14 (Markdown)	**Cell 6 — Define the angle-based heuristic**
Theory / concept	The MVD asks for a simple rule: if the pole angle is positive, take action 1; otherwise take action 0. This is not machine learning. It is a human-designed rule that reacts to the most intuitive state variable. Positive pole angle means the pole leans one way. Negative or zero angle means the pole leans the other way. The rule is simple but ignores three other useful variables.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 15 (Code)	Cell 6 - DEFINE CARTPOLE HEURISTIC POLICY
Theory / concept	This step creates a simple rule-based policy. The rule uses pole angle as a signal and pushes in the direction that should help keep the pole upright.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The heuristic rule is created: if pole angle is positive, push right; otherwise push left. This is a simple human-designed control rule, not learned from data.

Notebook Cell 16 (Markdown)	Cell 7 — Evaluate random and heuristic policies over 30 episodes
Theory / concept	One episode is not enough to judge an RL policy because results can vary. This cell evaluates both policies over 30 episodes and compares mean, median, and best reward. This gives a more reliable comparison than a single run. Mean reward shows average performance. Median reward shows typical performance. Best reward shows the strongest episode.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 17 (Code)	Cell 7 - EVALUATE RANDOM VS HEURISTIC POLICY
Theory / concept	This step compares two policies fairly over multiple episodes. The important metric is not one lucky episode; it is average and median reward over repeated trials.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	Across evaluation episodes, the random policy had mean reward about 29.33 and median 26.5; the angle heuristic had mean reward about 43.53 and median 42.0. The heuristic performed better on average because it reacts to pole direction.

Notebook Cell 18 (Markdown)	Cell 8 — Visualise policy reward distributions
Theory / concept	This box plot compares the full reward distribution for random and heuristic policies. It is useful because it shows whether the heuristic is consistently better or only better in a few lucky episodes. Higher median means better typical performance. Wider spread means more unstable performance. Dots show individual episode rewards.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 19 (Code)	Cell 8 - VISUALISE POLICY REWARD DISTRIBUTIONS
Theory / concept	This step compares reward spread. A box plot helps show not only which policy is better on average but also which one is more consistent.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The distribution chart compares variability. A better policy should have fewer very short episodes and a higher median reward.

Notebook Cell 20 (Markdown)	Cell 9 — Visualise policy summary metrics
Theory / concept	This chart summarizes the random-vs-heuristic comparison using mean, median, and best reward. It gives a quick BI-style view of which policy performs better. Mean = average performance. Median = typical performance. Best = strongest episode.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 21 (Code)	Cell 9 - VISUALISE POLICY SUMMARY METRICS
Theory / concept	This step summarizes mean, median and best reward in a business-readable way. It is similar to comparing model KPIs in a BI dashboard.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The summary chart shows that the heuristic improves average survival compared with the random baseline, although it is still far from a solved CartPole agent.

Notebook Cell 22 (Markdown)	Cell 10 — Explain done=True
Theory / concept	In Gymnasium, an episode ends when either terminated=True or truncated=True . We often combine them into done = terminated or truncated . This tells the training loop to stop the current episode and reset the environment. terminated: the environment naturally ended. truncated: a time limit or cutoff ended the episode. done: either one happened.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 23 (Code)	Cell 10 - EXPLAIN DONE=TRUE
Theory / concept	This step clarifies when an episode stops. Gymnasium separates terminated and truncated; the old simple word done usually means one of these became true.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	In the executed example, the episode ended after 50 steps with total reward 50. The terminated flag was True and truncated was False, meaning the task ended because the environment failure condition occurred, not because the time limit was reached.

Intermediate: FrozenLake Q-Learning

Notebook Cell 24 (Markdown)	Section 2 — Intermediate MVD: Q-Learning on FrozenLake-v1
Theory / concept	FrozenLake is a small grid-world problem, so tabular Q-Learning is suitable. The agent stores one Q-value for every state-action pair and updates those values from experience.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 25 (Markdown)	Cell 11 — Create deterministic FrozenLake and initialise Q-table
Theory / concept	FrozenLake has 16 states and 4 actions. In the deterministic version, actions behave predictably. This makes it easier to see Q-Learning work before adding slippery stochastic transitions. State = current grid cell. Action = left, down, right or up. Q-table shape = states × actions.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 26 (Code)	Cell 11 - CREATE FROZENLAKE AND INITIAL Q-TABLE
Theory / concept	This step starts tabular RL. FrozenLake has discrete states and actions, so a Q-table can store one learned value for each state-action pair.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	FrozenLake was created with 16 states and 4 actions. The initial Q-table starts with zeros because the agent has not yet learned which action is useful in each state.

Notebook Cell 27 (Markdown)	Cell 12 — Define epsilon-greedy action selection
Theory / concept	Epsilon-greedy is the exploration strategy. With probability epsilon, the agent explores using a random action. Otherwise, it exploits by choosing the best action from the current Q-table. This prevents the agent from getting stuck too early. Explore = try something random. Exploit = use the best known action. Epsilon slowly decreases during training.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 28 (Code)	Cell 12 - DEFINE EPSILON-GREEDY ACTION SELECTION
Theory / concept	This step implements exploration versus exploitation. The agent must try unknown actions early but gradually trust learned Q-values later.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The decision rule is ready: explore with a random action when a random number is less than epsilon; otherwise exploit the best Q-table action.

Notebook Cell 29 (Markdown)	Cell 13 — Define Q-value update and training helpers
Theory / concept	The Q-Learning update changes one Q-table value after each experience. The agent compares the old estimate with a new target: immediate reward plus discounted future value. Over many episodes, useful actions receive higher Q-values. alpha controls update strength. gamma controls future-reward importance. rolling mean over 100 episodes tracks learning progress.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 30 (Code)	Cell 13 - DEFINE Q-LEARNING UPDATE HELPERS
Theory / concept	This step implements the core Bellman-style Q-learning update. The update improves one state-action value after each experience.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The Q-value update helper is ready. It moves the old Q-value toward the target: immediate reward plus discounted best future Q-value.

Notebook Cell 31 (Markdown)	Cell 14 — Define Q-Learning training function
Theory / concept	This function runs many FrozenLake episodes. In every step, the agent chooses an action with epsilon-greedy logic, observes the reward and next state, updates the Q-table, and continues until the episode ends. The MVD requires 10,000 episodes. The function returns Q-table, rewards and rolling reward. The same function is reused for deterministic and slippery FrozenLake.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 32 (Code)	Cell 14 - DEFINE Q-LEARNING TRAINING FUNCTION
Theory / concept	This step wraps the repeated learning process into a reusable function. It trains the Q-table, decays exploration and records reward history.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The reusable Q-learning training loop is ready. It trains over many episodes and records reward history plus rolling reward for evaluation.

Notebook Cell 33 (Markdown)	Cell 15 — Train deterministic FrozenLake for 10,000 episodes
Theory / concept	This cell completes the main Intermediate MVD requirement. The deterministic environment is easier because the same action has a predictable result. The final last-100 mean reward is interpreted as the recent goal-reaching success rate. Training length: 10,000 episodes. Reward 1 means the goal was reached. Final last-100 reward shows recent success rate.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 34 (Code)	Cell 15 - TRAIN DETERMINISTIC FROZENLAKE
Theory / concept	This step trains the agent in a predictable grid. Deterministic movement is a good first test because the same action always behaves the same way.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	After 10,000 episodes on deterministic FrozenLake, the final last-100 mean reward was about 0.94 and total successes were 7,655. This means the learned Q-table solved the predictable lake frequently near the end of training.

Notebook Cell 35 (Markdown)	**Cell 16 — Plot mean reward over last 100 episodes**
Theory / concept	This line chart is the required Q-Learning learning curve. A rising rolling reward means the Q-table is improving. A high flat line means the learned policy has become stable. MVD-required plot. Rolling window = 100 episodes. Higher curve means more frequent goal success.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 36 (Code)	**Cell 16 - PLOT DETERMINISTIC FROZENLAKE LEARNING CURVE**
Theory / concept	This step plots rolling mean reward. A rising rolling mean means the agent is reaching the goal more often over recent episodes.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The rolling reward curve rises over training. The final rolling mean was about 0.94, showing strong recent performance.

Notebook Cell 37 (Markdown)	**Cell 17 — Visualise learned FrozenLake policy**
Theory / concept	A Q-table is hard to read directly. This grid converts the best action in each state into an arrow. It shows what the trained agent would do from each grid cell. Each cell is one state. Each arrow is the best action. This makes the learned policy understandable.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 38 (Code)	**Cell 17 - VISUALISE LEARNED FROZENLAKE POLICY**
Theory / concept	This step converts Q-table values into arrows. The best action in each state becomes the learned policy.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The policy visual translates the Q-table into actions for each grid cell. This makes the learned route toward the goal easier to interpret than raw Q-values.

Notebook Cell 39 (Markdown)	Cell 18 — Test alpha and gamma combinations
Theory / concept	The MVD asks us to vary alpha and gamma. Alpha controls how aggressively new experience updates the Q-table. Gamma controls how much future reward matters. We measure convergence speed by finding when rolling reward first reaches 0.8. alpha values: 0.1, 0.5, 0.9. gamma values: 0.7, 0.9, 0.99. Fastest convergence = lowest episode number.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 40 (Code)	Cell 18 - ALPHA/GAMMA EXPERIMENT
Theory / concept	This step tests learning-rate and discount-factor sensitivity. Alpha controls how aggressively values update; gamma controls how much future reward matters.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The fastest combination in the executed run was alpha=0.1 and gamma=0.90, reaching the 0.8 rolling reward threshold around episode 2,408. All tested combinations eventually achieved strong final last-100 rewards between about 0.92 and 0.98.

Notebook Cell 41 (Markdown)	Cell 19 — Visualise alpha/gamma convergence speed
Theory / concept	This heatmap compares the full alpha/gamma experiment. It is easier to understand than a table because low and high convergence episodes are visible immediately. Rows show alpha. Columns show gamma. Lower values mean faster convergence.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 42 (Code)	Cell 19 - ALPHA/GAMMA CONVERGENCE HEATMAP
Theory / concept	This step makes parameter tuning easier to read. A heatmap highlights which alpha/gamma combinations reached good performance fastest.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The heatmap compares how quickly alpha/gamma combinations reached the convergence threshold. Lower episode numbers mean faster learning.

Notebook Cell 43 (Markdown)	Cell 20 — Visualise final reward by alpha/gamma setting
Theory / concept	Fast convergence is not the only thing that matters. This chart shows the final mean reward for each alpha/gamma setting. A good parameter combination should ideally learn quickly and finish with high reward. Final reward checks stability. Convergence checks speed. Both are useful for model selection.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 44 (Code)	Cell 20 - FINAL REWARD BY ALPHA/GAMMA SETTING
Theory / concept	This step checks final performance for each parameter setting. Fast learning is useful, but a final stable reward is also important.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The final reward chart checks that fast convergence is not the only goal. Some combinations may learn slowly but still finish with strong performance.

Notebook Cell 45 (Markdown)	Cell 21 — Train Q-Learning on slippery FrozenLake
Theory / concept	The slippery version introduces stochastic transitions. This means that the action chosen by the agent may not lead exactly where expected. The agent now learns in a noisier and less predictable environment. Deterministic: action result is predictable. Slippery: action result can vary. Stochastic environments usually learn more slowly.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 46 (Code)	Cell 21 - TRAIN SLIPPERY FROZENLAKE
Theory / concept	This step adds stochastic transitions. The same action may not lead where intended, so the Q-table receives noisier feedback.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The deterministic version reached about 0.94 final last-100 reward and 7,655 successes. The slippery version reached only about 0.20 final last-100 reward and 1,704 successes, showing that stochastic transitions make learning harder.

Notebook Cell 47 (Markdown)	**Cell 22 — Compare deterministic and slippery learning curves**
Theory / concept	This chart directly answers what happens with stochastic transitions. If the slippery curve is lower or more unstable, it means the agent has a harder time learning because actions do not always produce the expected movement. Line chart shows learning over time. Deterministic should be easier. Slippery adds uncertainty.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 48 (Code)	**Cell 22 - DETERMINISTIC VS SLIPPERY LEARNING CURVES**
Theory / concept	This step compares predictable and random-transition worlds. It shows how environment uncertainty affects learning progress.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The comparison curve shows that stochastic movement reduces predictability. Even a good action may move the agent sideways, so reward becomes less stable.

Notebook Cell 49 (Markdown)	**Cell 23 — Visualise final deterministic vs slippery comparison**
Theory / concept	This bar chart gives a simple final comparison of recent success rates. It is useful for presentation because it shows the stochastic-transition impact in one view. Higher bar = higher recent success rate. The comparison supports the MVD explanation. This is a BI-style summary chart.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 50 (Code)	**Cell 23 - FINAL DETERMINISTIC VS SLIPPERY SUMMARY**
Theory / concept	This step summarizes how much harder stochastic RL is. It converts learning-curve evidence into simple final KPIs.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The final bar-style summary confirms the gap between predictable and slippery environments. Deterministic FrozenLake is much easier for tabular Q-learning.

Expert: DQN and Double DQN on CartPole

Notebook Cell 51 (Markdown)	**Section 3 — Expert MVD: DQN and Double DQN on CartPole-v1**
Theory / concept	Tabular Q-Learning works for small state spaces. CartPole observations are continuous numbers, so a Q-table is not practical. DQN uses a neural network to approximate Q-values from the state.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 52 (Markdown)	**Cell 24 — Define DQN configuration**
Theory / concept	The MVD requires a DQN network with architecture 4 64 64 2, replay buffer size 10,000, and target network update every 100 steps. This cell defines those settings explicitly. 4 inputs = CartPole observation variables. 2 outputs = action Q-values. Replay buffer and target network stabilize learning.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 53 (Code)	**Cell 24 - DQN CONFIGURATION**
Theory / concept	This step defines the deep RL experiment. DQN replaces the Q-table with a neural network because CartPole observations are continuous.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The DQN settings match the MVD: architecture 4 to 64 to 64 to 2, replay buffer size 10,000, target network update every 100 steps, smoothing window 50 episodes and solved threshold above 195 mean reward.

Notebook Cell 54 (Markdown)	**Cell 25 — Define the DQN neural network**
Theory / concept	The network receives the four CartPole observation values and outputs two Q-values. The larger Q-value indicates the action the model currently believes will give higher future reward. Layer 1: 4 64. Layer 2: 64 64. Output: 64 2.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 55 (Code)	**Cell 25 - DQN NETWORK**
Theory / concept	This step creates the neural approximator for Q-values. The network maps four state variables to two action values.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The network architecture is created with four CartPole inputs, two hidden layers of 64 units and two output Q-values. This satisfies the MVD architecture requirement.

Notebook Cell 56 (Markdown)	**Cell 26 — Define replay buffer**
Theory / concept	Experience replay stores past transitions. Instead of learning only from the most recent step, the model samples a random mini-batch from memory. This improves stability because it mixes old and new experiences. Transition = state, action, reward, next state, done. Capacity = 10,000. Random mini-batches reduce correlation.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 57 (Code)	**Cell 26 - REPLAY BUFFER**
Theory / concept	This step creates memory. The agent stores past transitions and later learns from random mini-batches instead of only the latest step.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The replay buffer is created with capacity 10,000 transitions. It starts empty and fills during training with state-action-reward-next-state experiences.

Notebook Cell 58 (Markdown)	**Cell 27 — Define DQN action and tracking helpers**
Theory / concept	This cell creates helper functions for action selection, reward smoothing, and threshold detection. These helpers keep the training code easier to read. Action selection uses epsilon-greedy logic. Moving average smooths reward over 50 episodes. Threshold detection answers when mean reward exceeds 195.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 59 (Code)	Cell 27 - DQN ACTION AND TRACKING HELPERS
Theory / concept	This step defines support tools for deep RL training: action selection, moving averages and solved-threshold detection.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The helper functions are ready for epsilon-greedy action selection, moving-average calculation and solved-threshold detection.

Notebook Cell 60 (Markdown)	Cell 28 — Define DQN optimisation step
Theory / concept	The optimisation step is the neural-network version of the Q-Learning update. It compares current Q-values with target Q-values and updates the model weights. Standard DQN and Double DQN differ in how the next-state target is calculated. DQN: target network chooses max next Q-value. DDQN: policy network chooses action, target network evaluates it. This helps reduce overestimation in Double DQN.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 61 (Code)	Cell 28 - DQN OPTIMISATION STEP
Theory / concept	This step trains the neural network. It compares predicted Q-values with target Q-values and updates the model using loss and backpropagation.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The optimisation step is ready. It samples from replay memory, computes current and target Q-values, calculates loss and updates the neural network weights.

Notebook Cell 62 (Markdown)	Cell 29 — Define DQN/DDQN training loop
Theory / concept	This function trains either DQN or Double DQN. It repeatedly runs CartPole episodes, stores transitions, trains from replay memory, and updates the target network every 100 steps. Same function handles DQN and DDQN. Reward history is saved for charts. Solved threshold is checked from the 50-episode moving average.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 63 (Markdown)	**Cell 29A — Define one DQN/DDQN episode runner**
Theory / concept	To keep the notebook readable, the deep RL training logic is split into two parts. This cell runs one CartPole episode: it selects actions, stores transitions, trains from replay memory, updates the target network every 100 steps, and returns the episode reward. The next cell repeats this episode runner many times. This reduces one large code block into smaller learning steps. The function works for both DQN and Double DQN. The same replay buffer and target-network logic is preserved.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 64 (Code)	**Cell 29A - ONE DQN/DDQN EPISODE RUNNER**
Theory / concept	Define one episode runner. Reset environment at the start of the episode. Start episode reward. Start episode loss list. Start done flag. Start local step counter. Run the episode until the environment ends or the safety step cap is reached. Select action using epsilon-greedy DQN logic. Apply action in CartPole. Increase episode step count. Build done flag with controlled step cap. Store transition in replay memory. Optimise policy network once from replay memory. Store loss when optimisation...
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The output should be interpreted in the notebook immediately after execution. The result supports the MVD workflow and the following chart or conclusion cell.

Notebook Cell 65 (Code)	**Cell 29B - DQN/DDQN TRAINING LOOP**
Theory / concept	This step controls full episode training. It links environment interaction, replay buffer storage, model updates and target-network synchronization.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The training loop is prepared to train either standard DQN or Double DQN. It handles episodes, replay storage, optimisation, epsilon decay, target-network updates and reward tracking.

Notebook Cell 66 (Markdown)	**Cell 30 — Train standard DQN**
Theory / concept	This cell trains the standard DQN agent on CartPole. The important MVD output is whether the 50-episode mean reward crosses 195 and, if so, at which episode. The training run is controlled for notebook stability. Results can vary because RL is stochastic. The notebook reports the actual result honestly.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 67 (Code)	**Cell 30 - TRAIN STANDARD DQN**
Theory / concept	This step trains the standard DQN agent. Standard DQN uses a max operation from the target network to estimate next-state value.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	In the controlled executed run, standard DQN trained for 80 episodes, reached best episode reward 169, final 50-episode mean 17.55, and did not reach the >195 solved threshold.

Notebook Cell 68 (Markdown)	**Cell 31 — Train Double DQN**
Theory / concept	Double DQN changes how target Q-values are calculated. It uses the policy network to choose the next action and the target network to evaluate that action. This can reduce overoptimistic Q-value estimates. Same architecture as DQN. Same replay buffer and target update logic. Comparison is based on the same metrics.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 69 (Code)	**Cell 31 - TRAIN DOUBLE DQN**
Theory / concept	This step trains Double DQN. DDQN separates action selection from action evaluation to reduce overestimated Q-values.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	In the controlled executed run, Double DQN trained for 80 episodes, reached best episode reward 336, final 50-episode mean 46.23, and did not reach the >195 solved threshold within the configured limit.

Notebook Cell 70 (Markdown)	**Cell 32 — Visualise raw DQN/DDQN rewards**
Theory / concept	Raw rewards show episode-by-episode volatility. In deep RL, raw rewards can jump up and down because the agent is still exploring and the neural network keeps changing. Raw curve shows instability. Smoothed curve is better for judging progress. Large spikes can happen before stable learning.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 71 (Code)	Cell 32 - RAW DQN/DDQN REWARD CURVES
Theory / concept	This step visualizes raw episode rewards. Raw curves expose volatility and lucky or unlucky episodes.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The raw reward curves show episode-to-episode volatility. RL training is noisy, so raw curves are useful but not enough for stable judgement.

Notebook Cell 72 (Markdown)	Cell 33 — Visualise smoothed DQN/DDQN learning curves
Theory / concept	This is the main Expert MVD chart. It plots the reward smoothed over 50 episodes and marks the CartPole solved threshold of mean reward greater than 195. Smoothed reward shows stable trend. Dashed line marks the solved threshold. If the model reaches the line, the episode is marked.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 73 (Code)	Cell 33 - SMOOTHED DQN/DDQN LEARNING CURVES
Theory / concept	This step visualizes the required 50-episode smoothed reward. Smoothing is used because RL reward is noisy.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The smoothed 50-episode curves compare learning stability. Neither agent crossed the >195 threshold in the controlled run, but Double DQN showed stronger best reward and final smoothed reward.

Notebook Cell 74 (Markdown)	Cell 34 — Compare final DQN and DDQN results
Theory / concept	This table compares the final deep-RL agents using best single episode, final smoothed reward, and solved-threshold status. This directly answers the MVD comparison task. Best episode = peak performance. Final 50-episode mean = recent stability. Threshold status = whether mean reward >195 was reached.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 75 (Code)	Cell 34 - FINAL DQN/DDQN COMPARISON TABLE
Theory / concept	This step compares final performance using best reward, final smoothed reward and solved-threshold status.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The final comparison showed DQN best reward 169 and final 50-episode mean 17.55, while Double DQN best reward was 336 and final 50-episode mean 46.23. Neither reached the solved threshold in the short controlled run.

Notebook Cell 76 (Markdown)	Cell 35 — Visualise final DQN/DDQN summary
Theory / concept	This bar chart is the final BI-style summary for the Expert section. It shows recent stable reward for each agent and the solved threshold line. Bars show final 50-episode mean reward. Dashed line shows >195 solved threshold. This chart is easy to use in presentation or documentation.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 77 (Code)	Cell 35 - FINAL DQN/DDQN SUMMARY CHART
Theory / concept	This step turns final DQN/DDQN metrics into a simple visual comparison with the 195 solved threshold.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The final chart compares the final smoothed reward against the CartPole solved threshold. It makes clear that the agents improved but did not meet the stable >195 target under the limited episode setting.

Final checklist and conclusion

Notebook Cell 78 (Markdown)	Final Section — MVD coverage checklist
Theory / concept	The final table maps every MVD requirement to notebook evidence. This keeps the assignment coverage transparent and easy to verify.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 79 (Markdown)	Cell 36 — Final MVD checklist
Theory / concept	This final cell confirms that the notebook covers Beginner, Intermediate and Expert MVD tasks. It is the quality gate for assignment submission. Every required task is mapped to notebook evidence. The checklist also shows that the DQN threshold is tracked honestly. No CSV files are needed because RL data is generated by environment interaction.
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

Notebook Cell 80 (Code)	Cell 36 - FINAL MVD COVERAGE CHECKLIST
Theory / concept	This step proves assignment coverage. Every MVD requirement is mapped to notebook evidence.
What we are doing	The code implements this step using small, commented blocks so that the data flow is visible: create or update variables, run the environment or model, then display a table or chart.
Result / meaning	The final checklist maps every Beginner, Intermediate and Expert MVD requirement to notebook evidence. It confirms that the MVD tasks are covered.

Notebook Cell 81 (Markdown)	Final notebook conclusion
Theory / concept	This notebook completes the Learning Day 13 MVD assignment. The Beginner section explains Gymnasium environment interaction through CartPole. The Intermediate section implements tabular Q-Learning on FrozenLake, compares alpha/gamma settings, and shows why slippery stochastic transitions are harder. The Expert section implements DQN and Double DQN with PyTorch, including the required network architecture, replay buffer, target network update, smoothed learning curve and threshold tracking. The notebook is designed for submission and later revision: each step explains what is happening, why it...
What we are doing	This cell explains the upcoming notebook step, introduces the context and prepares the reader before code is run.
Result / meaning	No code output is produced. The value of this cell is conceptual clarity and notebook readability.

4. Complete glossary

The glossary explains the important Reinforcement Learning and notebook terms in simple language.

Term	Meaning
Action	A move chosen by the agent. In CartPole, action 0 pushes left and action 1 pushes right. In FrozenLake, actions move around the grid.
Agent	The learner or decision-maker. It observes the environment, chooses actions and tries to collect more reward over time.
Alpha (learning rate)	Controls how strongly a new experience changes the old Q-value. Low alpha learns slowly; high alpha reacts strongly to new information.
Bellman idea	A value is built from immediate reward plus discounted future value. This is the logic behind Q-Learning and DQN targets.
CartPole-v1	A Gymnasium control environment where a cart must balance a pole. It is used for random policy, heuristic policy, DQN and DDQN.
Convergence	The point where learning becomes stable and recent performance stays high instead of changing randomly.
Deep Q-Network (DQN)	A Q-Learning method where a neural network predicts Q-values instead of using a Q-table.
Discount factor (gamma)	Controls how much future reward matters. A high gamma means the agent cares more about long-term reward.
Done	A simplified way to say the episode has ended. In Gymnasium this comes from terminated or truncated.
Double DQN (DDQN)	A DQN improvement that uses one network to choose the next action and another network to evaluate it, reducing overestimated Q-values.
Episode	One complete run of the environment from reset until termination or truncation.
Environment	The world the agent interacts with. It receives actions and returns observations, rewards and end signals.
Epsilon	The probability of exploring with a random action in epsilon-greedy action selection.
Epsilon decay	Gradually reducing epsilon so the agent explores more early and exploits learned knowledge later.
Epsilon-greedy	A policy that explores randomly with probability epsilon and otherwise chooses the best known action.
Experience tuple	One transition collected from interaction: state, action, reward, next state and done flag.
Exploration	Trying actions to gather information, even if they may not currently look best.
Exploitation	Choosing the action that currently looks best according to the policy or Q-values.
FrozenLake-v1	A small grid-world environment where the agent tries to reach a goal without falling into holes.
Gamma	Another name for the discount factor. It determines how much future reward contributes to the current value.
Gymnasium	A Python library that provides reinforcement learning environments such as CartPole and FrozenLake.

Term	Meaning
Heuristic policy	A human-designed rule. In the notebook, the CartPole heuristic uses pole angle to choose left or right.
Learning curve	A chart showing reward over episodes. It helps show whether the agent is improving.
Markov Decision Process (MDP)	A formal structure for RL problems with states, actions, transition probabilities, rewards and discounting.
Mean reward	Average total reward over multiple episodes. It is more stable than judging one episode.
Moving average	A smoothed average over recent episodes, used to reduce noise in RL charts.
Observation	The information returned by the environment. In CartPole, it contains four physical measurements.
Off-policy learning	Learning about a greedy or optimal policy while behavior may still include exploration. Q-Learning is off-policy.
On-policy learning	Learning the value of the same policy the agent is currently following. SARSA is a common example.
Policy	The rule or model that maps observations/states to actions.
Q-state	A state-action pair (s, a). It means being in a state and choosing a specific action.
Q-table	A table storing estimated Q-values for each state-action pair. It works when states and actions are discrete and small.
Q-value	The expected future reward for taking a specific action in a specific state.
Q-Learning	A model-free RL algorithm that learns Q-values directly from experience using the Bellman update.
Replay buffer	Memory used by DQN to store past transitions. The model trains on random mini-batches from this memory.
Reward	Feedback from the environment after an action. The agent tries to maximize total reward.
Reward threshold	A target performance level. For CartPole, mean reward above 195 is commonly treated as solved.
SARSA	An on-policy TD control method related to Q-Learning. It updates using the next action actually chosen by the current policy.
Slippery FrozenLake	A stochastic FrozenLake version where the intended action may not happen exactly, making learning harder.
State	The situation the environment is in. In FrozenLake it is a grid location; in CartPole it is described by continuous observation values.
Stochastic transition	A transition with randomness. The same action can lead to different next states.
Target network	A second neural network in DQN used to compute more stable target Q-values.
Temporal Difference learning	Learning from each step by comparing current estimates with a bootstrapped target from the next state.
Terminated	Gymnasium flag meaning the environment ended because the task reached a natural ending or failure condition.

Term	Meaning
Truncated	Gymnasium flag meaning the episode stopped because of an external limit, such as maximum time steps.